

A Project Report on

"CINEVISION / CINESAGA"

Online Cinema Ticket Booking System Based on Microservices

In partial fulfilment of the requirement for the
Award of the degree of
MASTERS IN COMPUTER APPLICATIONS
(2024 - 2026)

Under the supervision of: Dr. Pooja Mittal Associate Professor DCSA, M.D. University, Rohtak	Submitted by: Sonu Class: MCA 4th Semester Exam Roll No. 4402137
--	--

Department of Computer Science & Applications
Maharshi Dayanand University, Rohtak - 124001

CERTIFICATE-cum-DECLARATION

I, **SONU**, hereby declare that the work presented in this Project Report titled "**CINEVISION / CINESAGA**" and submitted as part of MCA 4th Semester to the Department of Computer Science & Applications, M.D. University, Rohtak, is an authentic record of my work carried out during the semester under the supervision of **Dr. Pooja Mittal**.

I further undertake that the matter embodied in this project report is my own work and has not been submitted by me or by any other candidate for the award of any other degree anywhere else.

(Signature of the student)

Sonu

MCA 4th Semester

Exam Roll No. 4402137

Countersigned by Internal Supervisor

Name: Dr. Pooja Mittal

Designation: Associate Professor, DCSA

Forwarded by:

Prof. Preeti Gulia

Head of Department

Acknowledgment

I would like to express my sincere gratitude to my internal supervisor, **Dr. Pooja Mittal**, for her valuable guidance, support, and encouragement throughout the development of this project. Her suggestions helped me understand the technical and practical aspects of building a complete microservice-based web application.

I also extend my thanks to **Prof. Preeti Gulia**, Head of the Department of Computer Science & Applications, for providing an academic environment that supports learning, experimentation, and project-based growth.

Finally, I am thankful to my teachers, classmates, and family members for their constant support and motivation. Their encouragement helped me complete this project report and the CineVision/CineSaga application successfully.

(Signature of the student)

Sonu

MCA 4th Semester

Exam Roll No. 4402137

Abstract

CineVision, implemented as CineSaga in the application, is a full-stack online cinema ticket booking system developed using a microservice architecture. The system allows users to browse movies, view detailed information, choose city and theatre, select showtime and seats, complete ticket booking, and receive a ticket confirmation through email. Administrators can manage movies, actors, directors, categories, cities, theatres, users, and platform activity through role-protected interfaces.

The backend is built with Java 21, Spring Boot, Spring Cloud, Spring Security, Eureka service discovery, API Gateway routing, MongoDB, Kafka, Redis, Java Mail Sender, and Micrometer/Zipkin tracing. The frontend is built with React, Redux, Axios, React Router, Bootstrap, and custom CSS. The project demonstrates how a real-world online ticketing platform can be decomposed into independent services while keeping the user journey simple and smooth.

This report describes the problem addressed by the system, its objectives, scope, design, implementation, technology stack, selective source code, user interface, testing approach, challenges, solutions, lessons learned, and future scope.

Table of Contents

Chapter 1 - Introduction

Chapter 2 - System Design

Chapter 3 - Technologies Used

Chapter 4 - Implementation

Chapter 5 - Selected Source Code

Chapter 6 - User Interface

Chapter 7 - Testing

Chapter 8 - Technical Challenges

Chapter 9 - Solutions

Chapter 10 - Lessons Learned

Chapter 11 - Future Scope

Conclusion

Bibliography

Chapter 1 - Introduction

1.1 Background

Cinema ticket booking has moved from counter-based reservation to online platforms where users expect quick discovery, seat availability, secure booking, and immediate confirmation. A modern cinema application must serve customers, administrators, and theatre operators while coordinating several backend responsibilities such as authentication, movie management, booking, payment records, ticket delivery, and notifications.

CineVision/CineSaga addresses this need through a browser-based booking platform supported by a distributed backend. Instead of creating a single monolithic server, the system is divided into services for users, movies, email, service discovery, and gateway routing.

1.2 Problem Statement

Manual or weakly connected booking systems create problems such as slow seat selection, duplicate booking risk, poor customer confirmation, limited admin control, and difficult maintenance. Users need a simple booking experience, while administrators need secure tools to manage movie and theatre data. The project solves this by offering a microservice-based cinema booking system with controlled roles, seat validation, ticket confirmation, and separate service responsibilities.

1.3 Motivation

The motivation behind this project is to understand how a production-inspired full-stack application can be built using modern enterprise technologies. Cinema booking is a useful domain because it naturally includes user identity, catalog browsing, seat selection, payment flow, asynchronous email confirmation, and administrative content management.

1.4 Purpose

The purpose of CineVision/CineSaga is to provide an efficient online platform where customers can book cinema tickets and administrators can manage cinema content. The project also demonstrates the practical use of microservices, service discovery, gateway routing, authentication, database persistence, caching, and frontend state management.

1.5 Scope

The project covers movie browsing, movie detail viewing, user registration and login, JWT-based role management, ticket count selection, seat selection, payment-detail handling, ticket record creation, email ticket generation, booking history, cancellation, favourite movies and theatres, wallet support, admin management, and distributed routing through an API gateway.

The project does not include a real third-party payment gateway or production deployment pipeline. Payment is represented through local booking logic suitable for academic demonstration.

1.6 Objectives

- To provide an easy-to-use online cinema ticket booking platform.
- To allow customers to browse movies, select showtimes, choose seats, and book tickets.
- To protect user and admin functionality using authentication and role-based authorization.
- To generate booking codes, QR-based tickets, and confirmation emails.
- To manage movies, actors, directors, categories, cities, theatres, and users through admin workflows.

- To demonstrate a maintainable microservice architecture using Spring Cloud and React.

1.7 Key Features

- Movie listing and detail pages with cast, director, category, and theatre information.
- User registration, OTP verification, login, Google login, and JWT session handling.
- Seat selection with booked-seat validation.
- Ticket booking, cancellation window, profile booking history, and downloadable ticket image.
- Email ticket delivery with QR code and visible ticket ID.
- Admin and theatre-manager controls for movies, theatres, users, and notifications.
- Service discovery, gateway routing, tracing, caching, and containerized infrastructure support.

1.8 Target Audience

Customers: Users who want to browse movies, select seats, book tickets, and manage their booking history.

Administrators: Users who manage movie content, users, theatres, and platform data.

Theatre Managers: Users assigned to theatre-management responsibilities.

Developers and Students: Learners studying microservices, Spring Boot, React, and distributed-system design.

1.9 Project Overview

The project contains five backend modules and one React frontend. The Eureka server registers services, the API gateway routes requests, the user service manages identity, the movie service manages cinema and booking workflows, and the email service handles ticket communication. The frontend connects to these services through a clean service layer.

Chapter 2 - System Design

2.1 Architectural Design

CineVision/CineSaga follows a microservice architecture. Each major responsibility is separated into a service so that the application remains maintainable and scalable. The frontend communicates with the API gateway, while backend services discover each other through Eureka.

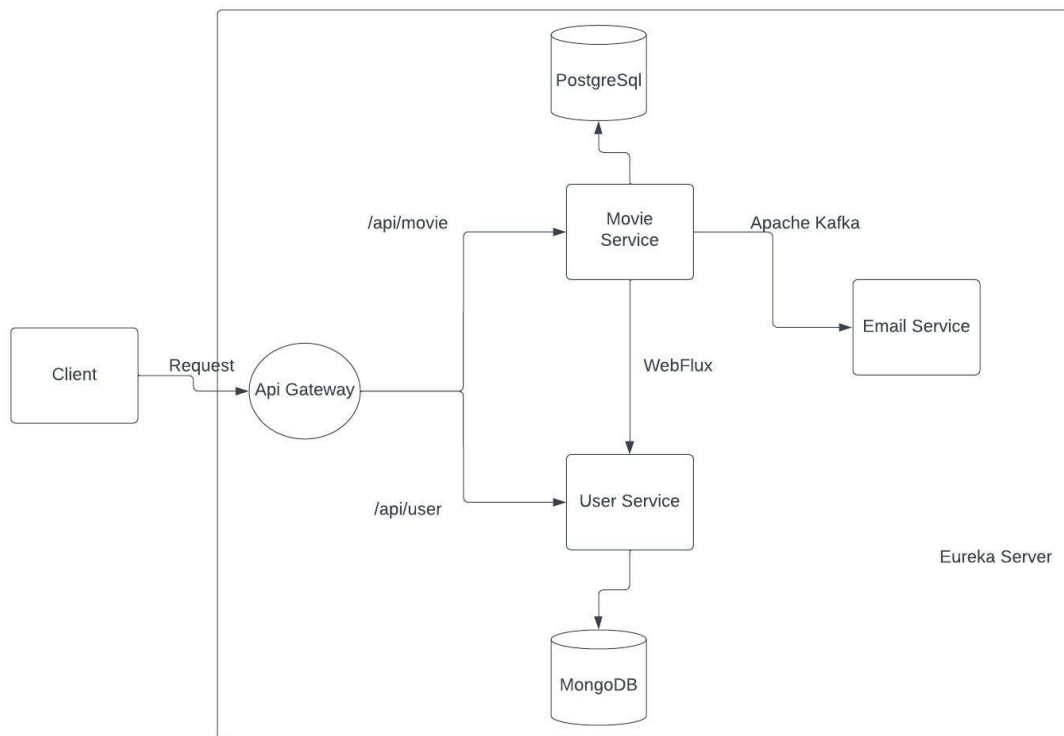


Figure 2.1: Microservice architectural design of CineVision/CineSaga

2.2 Main Modules

Module	Responsibility	Important Technology
Frontend	User interface, routing, booking screens, admin screens	React, Redux, Axios, Bootstrap
API Gateway	Single entry point and route mapping	Spring Cloud Gateway, Eureka Client
Eureka Server	Service registration and discovery	Netflix Eureka, Spring Security
User Service	Authentication, profile, roles, wallet, favourites	Spring Security, JWT, MongoDB
Movie Service	Movie catalog, theatres, comments, booking, cancellation	Spring Boot, MongoDB, Kafka, Mail, ZXing
Email Service	Template-based email notifications	Kafka, Java Mail Sender, FreeMarker

2.3 Use Case Diagram Explanation

The main actors of the system are Visitor, Registered Customer, Admin, Theatre Manager, and Email Service. A visitor can browse public movie data and register. A registered customer can login, select showtime, select seats, book tickets, download tickets, cancel bookings within the allowed window, add favourites, and comment on movies. Admin users can manage movies, users, theatres, and notifications.

Actor	Major Use Cases
Visitor	Browse movies, view details, register, login
Customer	Book ticket, select seats, pay, download ticket, cancel ticket, comment, manage profile
Admin	Manage movies, categories, actors, directors, users, roles, notifications
Theatre Manager	Manage assigned theatre information and theatre-related operations
Email Service	Send ticket confirmation and notification emails

2.4 Detailed Use Case Descriptions

Use Case	Description	Primary Actor
Register Account	A visitor submits name, email, password, and OTP verification details to create an account.	Visitor
Login	The user enters credentials and receives a JWT token for protected requests.	Customer/Admin
Browse Movies	The user views currently displaying and upcoming movies on the frontend.	Visitor
View Movie Details	The user checks synopsis, category, director, cast, comments, city, theatre, and showtime data.	Visitor/Customer
Book Ticket	The customer selects ticket count, seats, payment method, and confirms the booking.	Customer
Generate Ticket	The system creates a booking code, QR payload, downloadable ticket image, and email ticket.	Movie Service
Cancel Ticket	The customer cancels a confirmed ticket within the allowed cancellation window.	Customer
Manage Movies	Admins create, update, and delete movie records and related metadata.	Admin
Manage Users	Admins promote, blacklist, delete, notify, or assign user roles.	Admin
Manage Theatre	Theatre managers handle theatre-related responsibilities assigned by admin.	Theatre Manager

2.5 Service Interaction Flow

- 1 The React frontend sends requests to the API gateway.
- 2 The gateway forwards the request to the correct backend service using route definitions.
- 3 The user service authenticates users and issues JWT tokens.

- 4 The movie service validates showtime and seat data, creates booking and payment records, and appends booking history to the user profile.
- 5 Ticket data is used to generate a QR-based ticket and confirmation email.
- 6 Eureka keeps service locations dynamic so services can run on different ports.

2.6 Booking Sequence

- 1 The customer opens a movie detail page and chooses theatre and showtime.
- 2 The frontend stores the selected movie context in Redux and navigates to the booking page.
- 3 The booking page asks the movie service for already-booked seats for the selected show.
- 4 The customer selects ticket counts and seats. The frontend prevents choosing more seats than tickets.
- 5 The payment form submits ticket information to the friendly gateway endpoint `/api/tickets/book`.
- 6 The gateway forwards the request to `/api/movie/payments/sendTicketDetail` in the movie service.
- 7 The movie service validates seat format, checks existing bookings, saves booking and payment metadata, and builds a QR payload.
- 8 The ticket email sender creates the PNG ticket and QR code, then sends it to the customer email address.
- 9 The frontend receives the booking response and shows the success page with downloadable ticket image.

2.7 Class and Service Diagrams

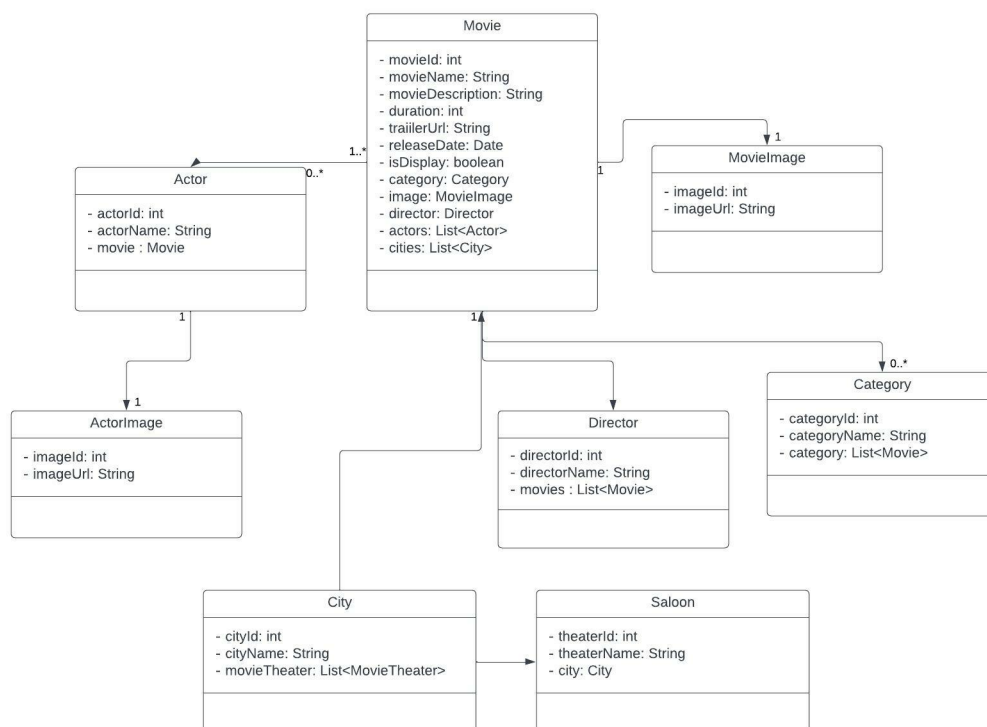


Figure 2.2: Movie service UML diagram

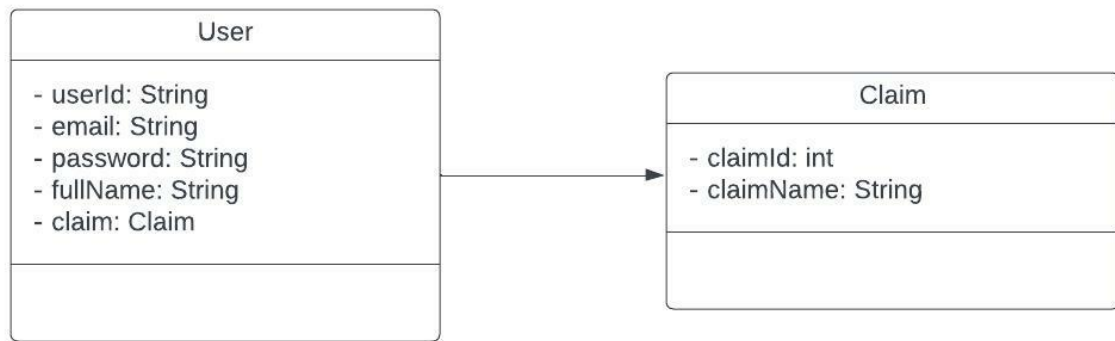


Figure 2.3: User service UML diagram

2.8 Database Design

The project uses MongoDB as the main persistence layer in the current codebase for user, movie, booking, profile, and related documents. The design stores user-specific snapshots such as booked movies and favourites in the user service, while movie and booking entities remain managed by the movie service.

Entity / Document	Main Fields	Purpose
User	userId, email, password, role, wallet, favourites, bookedMovies	Stores authentication and profile information
Movie	movieId, movieName, description, category, director, image	Stores catalog information
TicketBooking	bookingCode, movieName, saloonName, movieDay, seats, status	Stores confirmed bookings
PaymentCardDetail	paymentCode, booking, amount, maskedCardNumber, status	Stores payment metadata without exposing full card data
MovieSaloonTime	movie, saloon, showtime	Connects movies with theatre showtimes

2.9 Data Security Design

The system avoids exposing sensitive payment details directly. Card numbers are normalized, masked, and hashed with a secret before storage. User passwords are encoded through Spring Security password encoding. JWT tokens carry identity and authority claims, allowing protected service operations to verify the logged-in user without exposing passwords after login.

The project also uses role separation. Customer functions are separated from admin and theatre manager functions. This prevents ordinary users from accessing movie-management or user-management operations.

Chapter 3 - Technologies Used

3.1 Backend Technologies

Technology	Usage in Project
Java 21	Main backend language for all Spring services
Spring Boot 4.0.2	Application bootstrap, REST APIs, embedded servers
Spring Cloud 2025.1.1	Gateway routing, Eureka service discovery, cloud infrastructure
Spring Security	Authentication, authorization, endpoint protection
JWT	Portable authorization token between frontend and services
MongoDB	Document storage for users, movies, booking records, and related data
Kafka	Asynchronous message channel for email workflows
Java Mail Sender	Sends ticket and OTP emails
FreeMarker	Reusable email template generation
ZXing	Local QR generation for ticket images
Micrometer and Zipkin	Distributed tracing and observability
Docker Compose	Runs infrastructure components such as MongoDB, Kafka, Redis, and Zipkin

3.2 Frontend Technologies

Technology	Usage in Project
React 18	Component-based user interface
Redux	Shared state management for users and movies
React Router	Client-side navigation between pages
Axios	HTTP API calls through the gateway
Formik and Yup	Form handling and validation
Cleave.js	Formatted payment-card inputs
Bootstrap and CSS	Responsive visual layout and components
React Toastify	User feedback messages

3.3 Infrastructure

Docker Compose is used to run supporting infrastructure. The project configuration includes MongoDB, Mongo Express, Zookeeper, Kafka, Keycloak, Zipkin, and Redis. These tools make it possible to demonstrate a multi-service local environment from a single project directory.

3.4 Development Tools

The project is developed using Java 21, Maven 3.9.x, Node.js 22.x, npm 10.x, Git, and a JavaScript/Java-friendly IDE. Maven manages backend modules, while npm manages the React frontend dependencies.

3.5 Why These Technologies Were Selected

Spring Boot was selected because it reduces boilerplate and allows each service to run independently with an embedded server. This is suitable for microservice projects where each module should be buildable and runnable on its own.

Spring Cloud Gateway and Eureka were selected because a distributed system needs routing and service discovery. The gateway provides one entry point for the frontend, and Eureka removes the need to hard-code service ports.

React and Redux were selected because the frontend requires several pages sharing user state, movie state, and booking context. Redux keeps this shared state predictable across navigation.

MongoDB was useful because the project stores flexible documents such as users with favourites, wallet transactions, booking snapshots, and movie documents.

Kafka supports asynchronous workflows. Email delivery is naturally asynchronous because the booking should not depend on email rendering taking place inside the same request path.

ZXing was added for local QR generation so ticket emails can include real scannable QR codes even when external QR services are unavailable.

Chapter 4 - Implementation

4.1 Development Environment

Component	Selected Environment
Operating System	Windows 11
Backend Runtime	Java 21 LTS
Build Tool	Apache Maven 3.9.x
Frontend Runtime	Node.js 22.x and npm 10.x
Database	MongoDB through Docker/local runtime
Messaging	Kafka and Zookeeper
Editor	VS Code / Java IDE

4.2 Backend Implementation

The backend is divided into modules. Each module follows a layered pattern with controllers for request handling, business services for logic, repositories/DAOs for data access, DTOs for request and response transfer, and configuration classes for security, routing, messaging, and infrastructure.

- The user service handles authentication, OTP verification, profile, wallet, roles, favourites, and booking snapshots.
- The movie service handles movie catalog, theatre showtimes, comments, booking, payment records, ticket QR generation, and cancellation.
- The API gateway maps user-friendly frontend routes to internal service endpoints.
- The Eureka server allows services to register dynamically.
- The email service supports Kafka-based email template delivery.

4.3 Frontend Implementation

The frontend is organized into pages, layouts, services, store actions, reducers, and utility components. API integration is centralized in service classes. The booking page manages ticket count, seat selection, booked-seat checks, payment method, saved cards, and booking submission.

4.4 Booking Workflow

- 1 Customer logs in or registers.
- 2 Customer opens movie detail and selects theatre/showtime.
- 3 Booking page fetches already-booked seats.
- 4 Customer selects ticket counts and matching seats.
- 5 Frontend sends ticket information to the gateway.
- 6 Gateway routes the request to the movie service.
- 7 Movie service validates seats, creates booking and payment metadata, and generates the QR ticket payload.

- 8 Ticket email is generated and sent to the customer.
- 9 Frontend stores the last booking and shows the payment success page with download option.

4.5 API Endpoint Summary

Area	Frontend Route	Internal Service Route	Purpose
Auth	/api/auth/login	/api/user/auth/login	User login and JWT issue
Auth	/api/auth/signup/request-otp	/api/user/auth/signup/request-otp	Registration OTP request
Movies	/api/movies/now-showing	/api/movie/movies/displayingMovies	Load displaying movies
Movies	/api/movies/{movieId}	/api/movie/movies/{movieId}	Fetch movie details
Showtime	/api/movies/{movieId}/cities	/api/movie/cities/getCitiesByMovieId/{movieId}	Load cities for selected movie
Ticket	/api/tickets/booked-seats	/api/movie/payments/bookedSeats	Fetch unavailable seats
Ticket	/api/tickets/book	/api/movie/payments/sendTicketDetail	Create ticket booking
Ticket	/api/tickets/cancel	/api/movie/payments/cancel	Cancel confirmed ticket
Profile	/api/account/me	/api/user/users/me	Load customer profile
Admin	/api/admin/users/role	/api/user/users/admin/role	Change user role

4.6 Module-wise Implementation Notes

User Service: This service handles registration OTP, login, Google login, password reset OTP, user profile, role management, blacklisting, wallet, favourites, notification preferences, and booked movie snapshots.

Movie Service: This service contains the strongest business workflow. It validates selected seats, creates booking records, stores payment metadata, generates ticket QR payloads, communicates with the user service, and sends confirmation email tickets.

Gateway: The gateway provides friendly endpoint names for the frontend and maps them to internal service paths. This keeps the frontend cleaner and hides internal service-specific route names.

Frontend: React pages manage user interaction. The booking page coordinates ticket count, seat grid, booked-seat loading, saved card selection, UPI option, and final ticket submission.

Email Service: The email service supports Kafka-based template emails. In the current booking flow, the movie service also contains a direct ticket sender for PNG ticket attachments and QR generation.

Chapter 5 - Selected Source Code

Only important code excerpts are included in this report. Environment files, markdown files, generated target files, lock files, and large repetitive files are intentionally excluded. The snippets below represent the core logic of routing, authentication, booking, ticket QR generation, and frontend integration.

5.1 API Gateway Route for Ticket Booking

```
spring.cloud.gateway.server.webflux.routes[10].id=tickets-book
spring.cloud.gateway.server.webflux.routes[10].uri=lb://MOVIESERVICE
spring.cloud.gateway.server.webflux.routes[10].predicates[0]=Path=/api/tickets/book
spring.cloud.gateway.server.webflux.routes[10].filters[0]=SetPath=/api/movie/payments/sendTicketDetail

spring.cloud.gateway.server.webflux.routes[24].id=tickets-booked-seats
spring.cloud.gateway.server.webflux.routes[24].uri=lb://MOVIESERVICE
spring.cloud.gateway.server.webflux.routes[24].predicates[0]=Path=/api/tickets/booked-seats
spring.cloud.gateway.server.webflux.routes[24].filters[0]=SetPath=/api/movie/payments/bookedSeats
```

5.2 Axios Client with JWT Interceptor

```
export const apiClient = axios.create({
  baseURL: "http://localhost:8080/api"
});

apiClient.interceptors.request.use((config) => {
  const storedUser = sessionStorage.getItem("cineSagaUser");
  if (storedUser) {
    const user = JSON.parse(storedUser);
    if (user?.token) {
      config.headers.Authorization = `Bearer ${user.token}`;
    }
  }
  return config;
});
```

5.3 JWT Generation in User Service

```
public String generateToken(Authentication authentication) {
  List<String> authorities = authentication.getAuthorities().stream()
    .map(GrantedAuthority::getAuthority)
    .toList();

  return Jwts.builder()
    .claim("authorities", authorities)
    .setSubject(authentication.getName())
    .setIssuedAt(new Date())
    .setExpiration(java.sql.Date.valueOf(LocalDate.now().plusWeeks(2)))
    .signWith(Keys.hmacShaKeyFor(key.getBytes()))
    .compact();
}
```

5.4 Authentication Login Flow

```
public UserAuthenticationResponseDto login(UserLoginRequestDto request) {
  String email = request.getEmail().trim().toLowerCase(Locale.ROOT);
  User user = userService.getUserByEmail(email);
  ensureNotBlacklisted(user);

  Authentication auth = authenticationManager.authenticate(
    new UsernamePasswordAuthenticationToken(email, request.getPassword()));
  SecurityContextHolder.getContext().setAuthentication(auth);

  return UserAuthenticationResponseDto.builder()
    .userId(user.getUserId())
    .fullName(user.getFullName())
    .email(user.getEmail())
    .token(jwtProvider.generateToken(auth))
    .roles(auth.getAuthorities().stream()
      .map(GrantedAuthority::getAuthority).toList())
    .build();
}
```

5.5 Ticket Booking Controller

```
@PostMapping("sendTicketDetail")
public TicketBookingResponseDto sendTicketDetail(
    @Valid @RequestBody TicketInformationDto ticketInformationDto,
    @RequestHeader(value = "Authorization", required = false) String authorizationHeader) {
    return paymentService.sendTicketDetail(ticketInformationDto, authorizationHeader);
}

@GetMapping("bookedSeats")
public List<String> getBookedSeats(@RequestParam String movieName,
    @RequestParam String saloonName,
    @RequestParam String movieDay,
    @RequestParam String movieStartTime) {
    return paymentService.getBookedSeats(movieName, saloonName, movieDay, movieStartTime);
}
```

5.6 Core Booking and Ticket Response Logic

```
Set<String> seats = parseSeatNumbers(ticketInformationDto.getChairNumbers());
int requestedTickets = ticketInformationDto.getAdultTicketCount()
    + ticketInformationDto.getStudentTicketCount();
if (requestedTickets != seats.size()) {
    throw new IllegalArgumentException("Choose one seat for every ticket");
}

BigDecimal totalAmount = BigDecimal.valueOf(ticketInformationDto.getAdultTicketCount()).multiply(BigDecimal.valueOf(250))
    .add(BigDecimal.valueOf(ticketInformationDto.getStudentTicketCount()).multiply(BigDecimal.valueOf(150)));

booking.ensureBookingMetadata();
booking = ticketBookingDao.save(booking);

String chairNumbers = String.join(" ", seats);
String qrCodePayload = buildQrCodePayload(booking, chairNumbers);
ticketEmailSender.sendBookingTicket(booking, chairNumbers, totalAmount, qrCodePayload);

return TicketBookingResponseDto.builder()
    .status("SUCCESS")
    .bookingCode(booking.getBookingCode())
    .qrCodePayload(qrCodePayload)
    .recipientEmail(booking.getEmail())
    .build();
```

5.7 Local QR Generation for Email Ticket

```
private BufferedImage buildQrCodeBufferedImage(String qrCodePayload, int size)
    throws WriterException {
    Map<EncodeHintType, Object> hints = new EnumMap<>(EncodeHintType.class);
    hints.put(EncodeHintType.CHARACTER_SET, StandardCharsets.UTF_8.name());
    hints.put(EncodeHintType.ERROR_CORRECTION, ErrorCorrectionLevel.M);
    hints.put(EncodeHintType.MARGIN, 1);

    BitMatrix bitMatrix = new QRCodeWriter().encode(
        qrCodePayload, BarcodeFormat.QR_CODE, size, size, hints);
    BufferedImage qrImage = new BufferedImage(size, size, BufferedImage.TYPE_INT_RGB);
    for (int y = 0; y < size; y++) {
        for (int x = 0; x < size; x++) {
            qrImage.setRGB(x, y, bitMatrix.get(x, y)
                ? Color.BLACK.getRGB() : Color.WHITE.getRGB());
        }
    }
    return qrImage;
}
```

5.8 Frontend Payment Service

```
export class PaymentService {
    sendTicketDetail(ticketDetail) {
        return apiClient.post("/tickets/book", ticketDetail);
    }

    getBookedSeats(params) {
        return apiClient.get("/tickets/booked-seats", { params });
    }

    cancelTicket(bookingCode) {
        return apiClient.post("/tickets/cancel", { bookingCode });
    }
}
```

5.9 React Seat Selection Logic

```
function selectSeat(seatId) {
  if (bookedSeats.includes(seatId.toUpperCase())) {
    toast.warning("That seat is already booked. Please choose another one.");
    return;
  }
  setSelectedSeats(currentSeats => {
    if (currentSeats.includes(seatId)) {
      return currentSeats.filter(seat => seat !== seatId);
    }
    if (currentSeats.length >= totalTickets) {
      toast.warning("You already selected seats for every ticket.");
      return currentSeats;
    }
    return [...currentSeats, seatId];
  });
}
```

5.10 Ticket Booking Entity

```
@Document(collection = "bookings")
public class TicketBooking {
  @Id
  private String bookingCode;
  private String movieName;
  @Field("theaterName")
  private String saloonName;
  private String movieDay;
  private String movieStartTime;
  private String email;
  private String fullName;
  private BigDecimal totalAmount;
  private String status;
  private Instant createdAt;
  private List<TicketBookingSeat> seats = new ArrayList<>();

  public void ensureBookingMetadata() {
    if (bookingCode == null) {
      bookingCode = "CV-" + UUID.randomUUID().toString()
        .substring(0, 8).toUpperCase();
    }
    if (createdAt == null) {
      createdAt = Instant.now();
    }
  }
}
```

5.11 Booking Cancellation Logic

```
TicketBooking booking = ticketBookingDao.findByBookingCode(
  cancelTicketRequestDto.getBookingCode().trim());
if (booking == null) {
  throw new IllegalArgumentException("Booking was not found");
}
if ("CANCELLED".equalsIgnoreCase(booking.getStatus())) {
  throw new IllegalArgumentException("Ticket is already cancelled");
}
if (booking.getCreatedAt() != null
  && Instant.now().isAfter(booking.getCreatedAt()
    .plusSeconds(cancellationWindowMinutes * 60))) {
  throw new IllegalArgumentException("Tickets can be cancelled within "
    + cancellationWindowMinutes + " minutes of booking");
}
booking.setStatus("CANCELLED");
booking.setCancelledAt(Instant.now());
ticketBookingDao.save(booking);
```

5.12 Payment Card Masking and Hashing

```
private String maskCardNumber(String cardNumber) {
    if (cardNumber.length() < 4) {
        return "DUMMY PAYMENT";
    }
    String lastFour = cardNumber.substring(cardNumber.length() - 4);
    return "**** * * * * " + lastFour;
}

private String hashCardNumber(String hashSource) {
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    byte[] hashBytes = digest.digest((cardHashSecret + ":" + hashSource)
        .getBytes(StandardCharsets.UTF_8));
    StringBuilder hash = new StringBuilder();
    for (byte hashByte : hashBytes) {
        hash.append(String.format("%02x", hashByte));
    }
    return hash.toString();
}
```

5.13 Auth Controller OTP and Login Routes

```
@PostMapping({" /register/request-otp", " /signup/request-otp"})
public OtpRequestResponseDto requestRegistrationOtp(
    @Valid @RequestBody UserRegisterRequestDto request) {
    return userService.requestRegistrationOtp(request);
}

@PostMapping({" /register/verify-otp", " /signup/verify-otp"})
public UserProfileResponseDto verifyRegistrationOtp(
    @Valid @RequestBody VerifyRegistrationOtpRequestDto request) {
    return userService.verifyRegistrationOtp(request);
}

@PostMapping(" /login")
public UserAuthenticationResponseDto login(
    @Valid @RequestBody UserLoginRequestDto request) {
    return authService.login(request);
}
```

5.14 Downloaded Ticket QR Payload on Frontend

```
export function buildTicketQrPayload(booking) {
    if (booking?.qrCodePayload) {
        return booking.qrCodePayload;
    }

    return [
        "CineSaga Ticket",
        `Booking: ${booking.bookingCode || ""}`,
        `Movie: ${booking.movieName || ""}`,
        `Theatre: ${booking.salonName || booking.theatreName || ""}`,
        `Date: ${booking.movieDay || ""}`,
        `Time: ${booking.movieStartTime || booking.showtimeStartTime || ""}`,
        `Seats: ${booking.chairNumbers || booking.seats || ""}`,
        `Amount: ${formatCurrency(booking.totalAmount)}`
    ].join("\n");
}
```

5.15 React Profile Ticket Download Mapping

```
function toTicketImageBooking(bookedMovie) {
    return {
        bookingCode: getTicketCode(bookedMovie),
        movieName: getTicketMovieName(bookedMovie),
        salonName: getTicketTheaterName(bookedMovie),
        movieDay: bookedMovie?.movieDay,
        movieStartTime: bookedMovie?.showtimeStartTime || bookedMovie?.movieStartTime,
        chairNumbers: getTicketSeats(bookedMovie),
        totalAmount: bookedMovie?.totalAmount,
        recipientEmail: profile.email,
        qrCodePayload: bookedMovie?.qrCodePayload
    };
}
```

Chapter 6 - User Interface

The user interface focuses on a smooth customer journey and clear admin controls. The main customer path includes browsing movies, opening details, choosing showtime, selecting tickets and seats, completing payment, and viewing ticket confirmation.

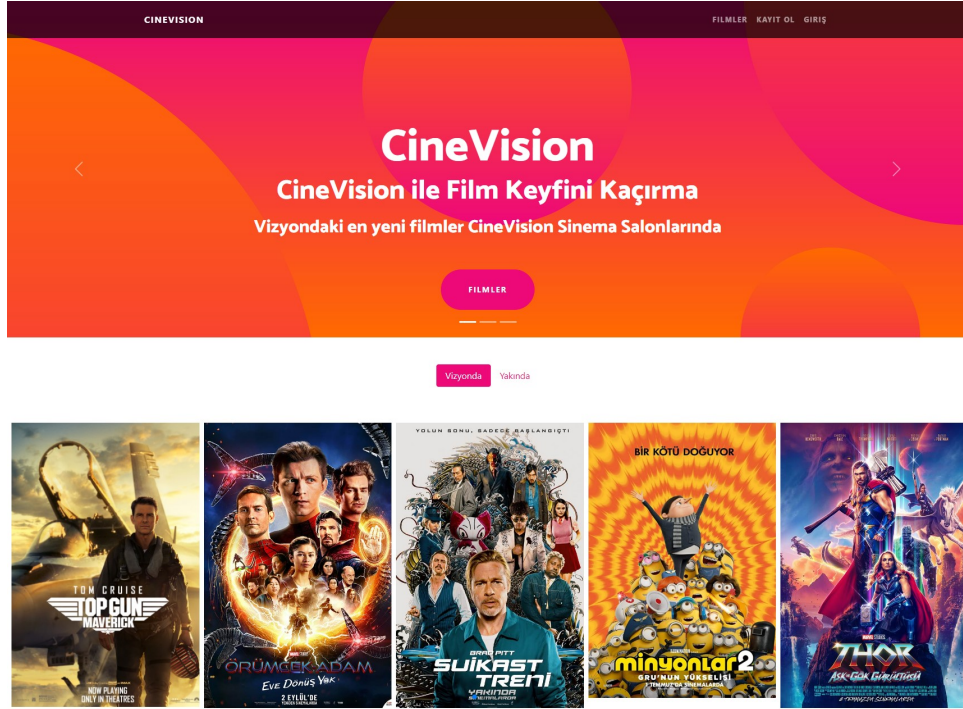


Figure 6.1: Main page showing cinema content

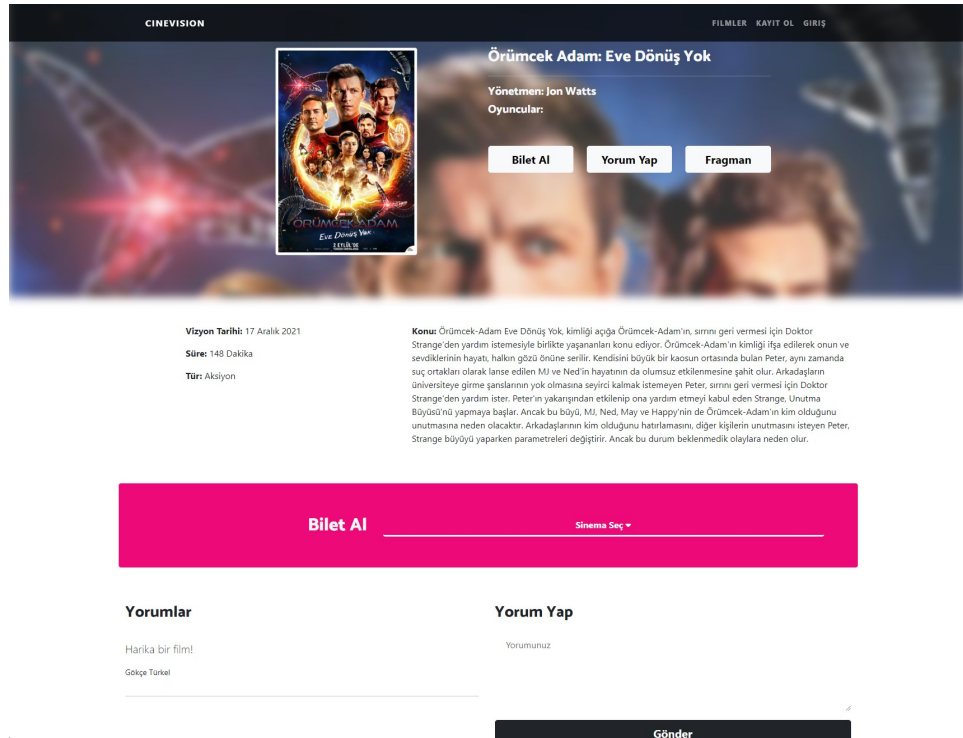


Figure 6.2: Movie detail page with showtime flow

CINEVISION
FILMLER KAYIT OL GİRİŞ

Örümcek Adam: Eve Dönüş Yok



17 Burda Ayrı
9 Ekim 2022
15.30

Biletini Seç
Devam Et

Film ve Seans seçiminin sonra bilet tipini seçmelisin. Eger öğrenciysen kimlik kartını yanında getirmeyi unutma.

TAM	Fiyat 25₺	-	2	+
ÖĞRENCİ	Fiyat 15₺	-	0	+

Toplam Tutar: 50.00 ₺

Koltuk Seç
Değiştir

Ödeme

Figure 6.3: Ticket and payment workflow

CINEVISION
FİLM EKLE FİLMLER ÇIKIŞ YAP

Film Ekle

Film bilgilerini eksiksiz olarak doldur ve filmin oyuncularını seçmek için ilerle.

Filmin İsmi

Filmin Özeti

Filmin Süresi

Vizyon Tarihi
99.aa.jyyy

Fragman Url

Kategori

Yönetmen

Yönetmen İsmi

☐ Film Vizyonda Mı?

Yönetmen yukarıdaki listede bulunmuyorsa lütfen yazın.

İleri

Figure 6.4: Admin movie management page

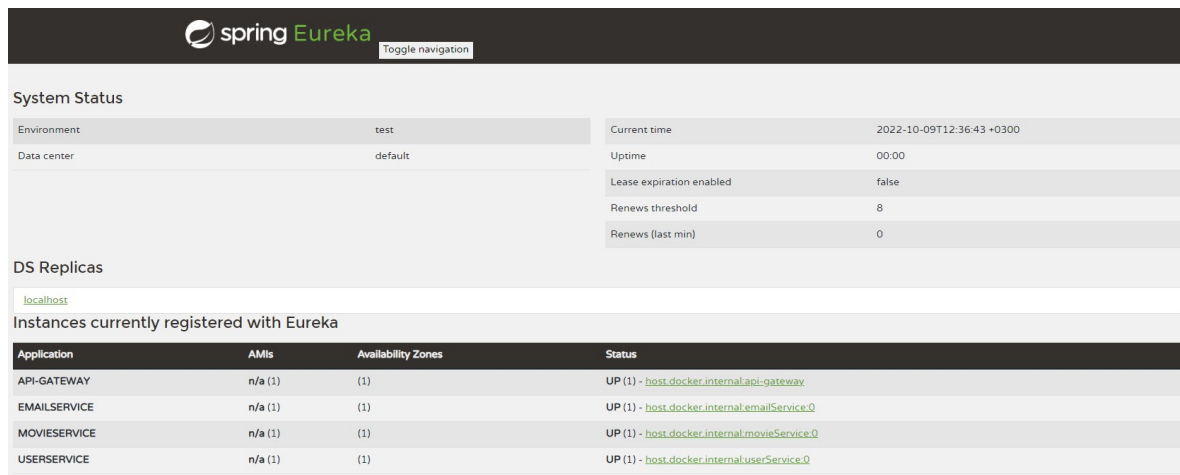


Figure 6.5: Eureka service registry panel

Chapter 7 - Testing

7.1 Tools Used for Testing

- Manual browser testing for end-to-end user flow.
- Maven compile/build checks for backend services.
- React production build for frontend verification.
- Postman or API client testing for backend endpoints.
- MongoDB/Mongo Express inspection for database records.
- Eureka dashboard for service registration checks.
- Email inbox testing for ticket confirmation and QR attachment.

7.2 Test Cases

Test Area	Test Case	Expected Result
Authentication	Register user and verify OTP	User account is created after valid OTP
Authentication	Login with valid credentials	JWT token and user profile are returned
Authorization	Open admin page as customer	Access is restricted
Movies	Load now-showing movies	Movie cards are displayed
Showtime	Select city and theatre	Available showtimes are listed
Booking	Select fewer seats than tickets	User receives validation warning
Booking	Select already-booked seat	Seat selection is blocked
Payment	Submit valid booking details	Booking code, QR payload, and success response are generated
Email	Receive ticket email	Email contains visible ticket ID and scannable QR ticket image
Cancellation	Cancel within allowed window	Booking status changes to CANCELLED
Profile	Open booking history	Booked tickets are shown to the signed-in user

Chapter 8 - Technical Challenges

- Coordinating multiple services through Eureka and API Gateway.
- Maintaining consistent JWT authorization across frontend and backend modules.
- Preventing duplicate seat booking during ticket selection.
- Keeping booking data synchronized between movie service and user profile snapshots.
- Generating email tickets with the same QR payload used by the website download.
- Designing a frontend flow that remains simple despite a distributed backend.
- Managing local infrastructure such as MongoDB, Kafka, Redis, and Zipkin through Docker.

Chapter 9 - Solutions

- Gateway routes were created to expose user-friendly frontend APIs while forwarding to internal service paths.
- JWT tokens are stored in session storage and added to every Axios request through an interceptor.
- Booked seats are checked before allowing selection and again before saving the booking.
- The movie service creates booking records and appends a booking snapshot to the user service.
- ZXing is used to generate real QR codes locally for email tickets, preventing dependence on external QR image services.
- Layered service structure keeps controller, business, DTO, and DAO responsibilities separate.
- Docker Compose centralizes the local infrastructure setup.

Chapter 10 - Lessons Learned

This project helped in understanding the practical difference between a monolithic application and a microservice system. It showed how service discovery, gateway routing, authentication, frontend state management, database design, and asynchronous communication fit together in one application.

It also highlighted the importance of small details in user experience. A ticket booking system is not complete until seat selection, payment confirmation, booking history, email delivery, and QR ticket download all behave consistently.

Chapter 11 - Future Scope

11.1 Feature Enhancements

- Integration with real payment gateways such as Razorpay or Stripe.
- Advanced theatre seat maps with multiple auditorium layouts.
- Movie recommendation system based on user interests and booking history.
- SMS and WhatsApp notifications in addition to email.
- Loyalty points, coupons, and membership plans.

11.2 Technical Improvements

- Stronger concurrency control for high-traffic seat booking.
- More automated unit, integration, and end-to-end tests.
- Container orchestration with Kubernetes for production deployment.
- Centralized logging and monitoring dashboards.
- Improved API documentation through OpenAPI/Swagger.

Conclusion

CineVision/CineSaga is a complete full-stack cinema ticket booking project that combines a React frontend with a Java Spring Boot microservice backend. The system supports movie discovery, authentication, booking, seat selection, payment metadata, ticket confirmation, QR ticket generation, profile management, admin workflows, service discovery, gateway routing, and observability.

The project demonstrates both functional application development and architectural understanding. It solves a practical user problem while showing how modern distributed web applications can be built in an organized, secure, and maintainable way.

Bibliography

- Spring Boot Documentation - Application development and REST services.
- Spring Cloud Documentation - Eureka, Gateway, and distributed service patterns.
- React Documentation - Component-based frontend development.
- MongoDB Documentation - Document database usage and persistence.
- Apache Kafka Documentation - Event-driven communication.
- ZXing Project Documentation - QR code generation.
- Report project source code: CineVisionMicroserviceProject repository.